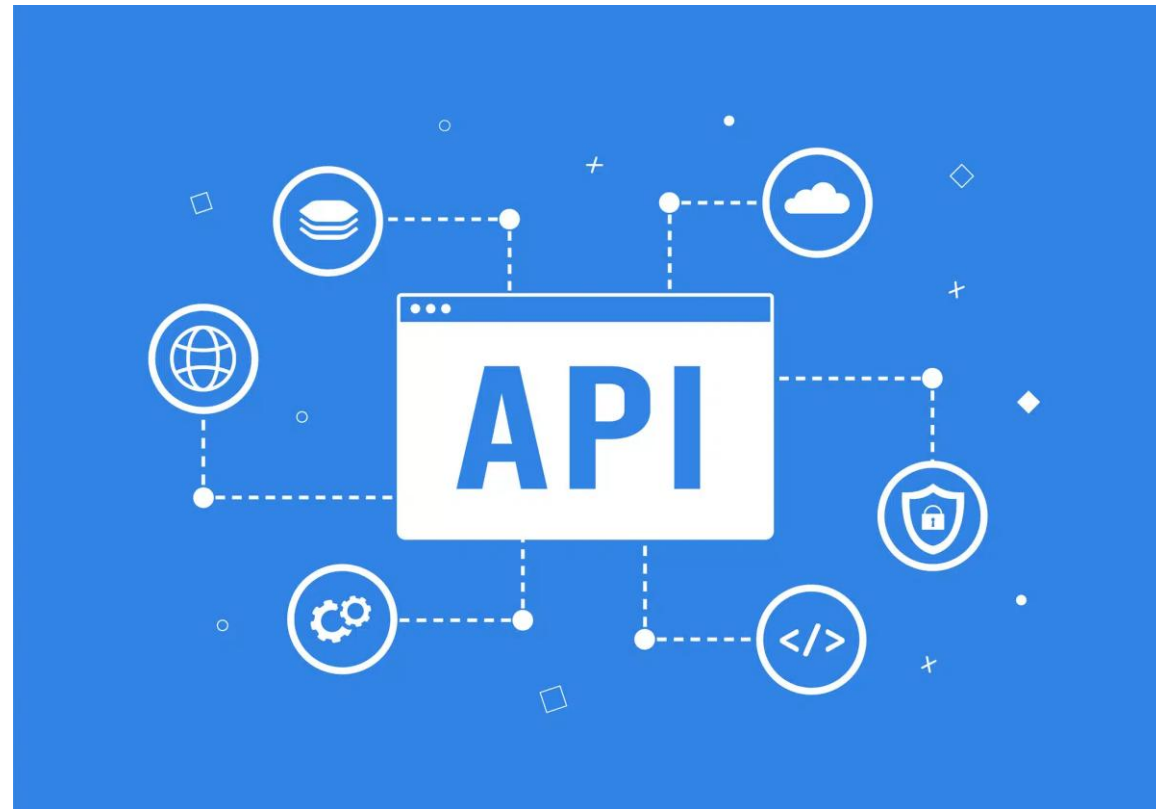


Interação com APIs

01

2026

SENAI - CRTI



Fundamentos de APIs e Protocolos

Como aplicações se comunicam na web

O que você vai aprender hoje

1

Serviços do Servidor

Reconhecer por que aplicações front-end precisam de serviços externos para funcionar.

2

HTTP/HTTPS

Entender o protocolo de comunicação que sustenta toda a web moderna.

3

Request / Response

Compreender o ciclo completo de uma requisição e sua resposta.

4

Papel das APIs

Identificar como APIs integram sistemas distintos em aplicações reais.

O Desafio da Aula

Contexto Real

Uma empresa está desenvolvendo um **sistema web de cadastro de clientes**. Ao digitar um CEP, o sistema deve preencher automaticamente endereço, bairro, cidade e estado — sem armazenar esses dados localmente.

Como uma aplicação web consegue buscar informações externas em tempo real?

O que precisamos entender

- Como o front-end obtém dados que não possui
- Quem fornece essas informações
- Qual protocolo viabiliza essa troca
- Como os dados chegam formatados ao navegador

 Esta situação representa um desafio real enfrentado por desenvolvedores no mercado de trabalho.

Aplicações que Você Usa Todos os Dias

App Bancário

Consulta saldo e extrato em servidores do banco em tempo real.

Delivery

Rastreia pedidos e atualiza status conectando ao servidor do restaurante.

E-commerce

Carrega catálogo de produtos, preços e estoque de uma fonte central.

CEP Automático

Preenche endereço completo consultando um serviço externo de geolocalização.



Mensagem-chave: Aplicações modernas dependem de serviços externos para funcionar. Nenhuma delas armazena todos os dados localmente.



O que é uma API?

API significa **Application Programming Interface** — uma interface que permite que dois sistemas se comuniquem, troquem dados e integrem funcionalidades sem expor sua lógica interna.

A Analogia do Restaurante

Você (cliente) não vai até a cozinha buscar sua comida. Você faz o **pedido ao garçom**, que leva à cozinha, e traz a **resposta** pronta para você.

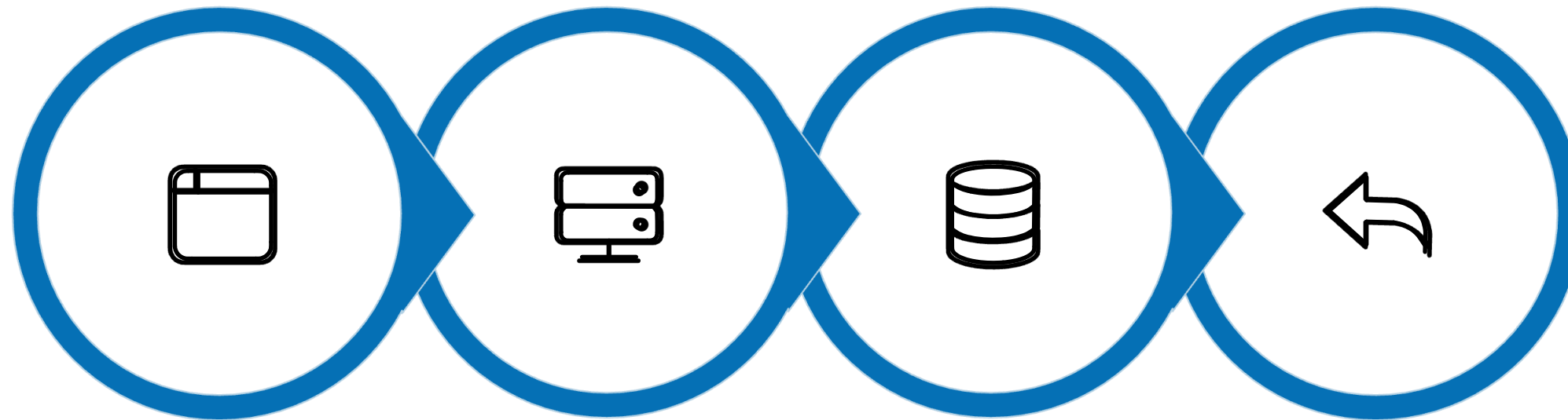
-  Cliente → quem solicita
-  Garçom → a API
-  Cozinha → o servidor/serviço
-  Prato → a resposta com dados

Na Prática Digital

- O front-end faz uma **solicitação** à API
- A API encaminha ao serviço correto
- O serviço processa e **retorna os dados**
- O front-end exibe o resultado ao usuário

APIs definem o **contrato de comunicação**: quais dados podem ser pedidos e em qual formato serão entregues.

Cliente × Servidor



Request

Processar

Consultar BD

Response

 Cliente (Front-end)

- Navegador web (Chrome, Firefox)
- Aplicativo Angular
- App mobile (iOS/Android)

Inicia a comunicação fazendo **requisições**. Exibe os dados ao usuário final.

 Servidor (Back-end)

- Recebe e interpreta a requisição
- Consulta banco de dados ou outros serviços
- Devolve uma **resposta estruturada**

Nunca é acessado diretamente pelo usuário — apenas pela camada de API.

HTTP – HyperText Transfer Protocol

O HTTP é o **protocolo de comunicação padrão da web**. Define as regras de como cliente e servidor trocam mensagens, incluindo estrutura, métodos, cabeçalhos e códigos de status.



Headers

Metadados da requisição: tipo de conteúdo, autenticação, idioma aceito.



Body

Corpo da mensagem com os dados enviados (usado em POST/PUT).



Método

Ação desejada: GET, POST, PUT, DELETE. Define a *intenção* da requisição.



Status

Código numérico que indica o resultado: 200 OK, 404 Not Found, 500 Error.



Métodos HTTP

Os métodos HTTP indicam ao servidor **qual operação deve ser executada**. Cada método corresponde a uma ação diferente sobre os recursos disponíveis.

1

GET

Buscar dados. Não altera nada no servidor. Ex: listar clientes, buscar CEP.

2

POST

Criar novo recurso. Envia dados no corpo. Ex: cadastrar um novo usuário.

3

PUT

Atualizar recurso. Substitui os dados existentes. Ex: editar cadastro

4

DELETE

Remover recurso. Exclui o item indicado. Ex: deletar um produto.

 Nesta aula, o foco é reconhecer os métodos. Nas próximas aulas iremos utilizá-los com Angular na prática.

HTTPS — HTTP + Segurança



O que é HTTPS?

É o HTTP com uma camada adicional de segurança fornecida pelo protocolo **TLS/SSL** (Transport Layer Security / Secure Sockets Layer). Toda a comunicação é **criptografada** entre cliente e servidor.

🔒 Criptografia dos dados em trânsito

✅ Integridade — dados não são alterados

🛡️ Confidencialidade — só destino lê

👤 Autenticidade — servidor verificado

Onde o HTTPS é obrigatório

🏦 Internet Banking

Dados financeiros e senhas nunca devem trafegar sem criptografia.

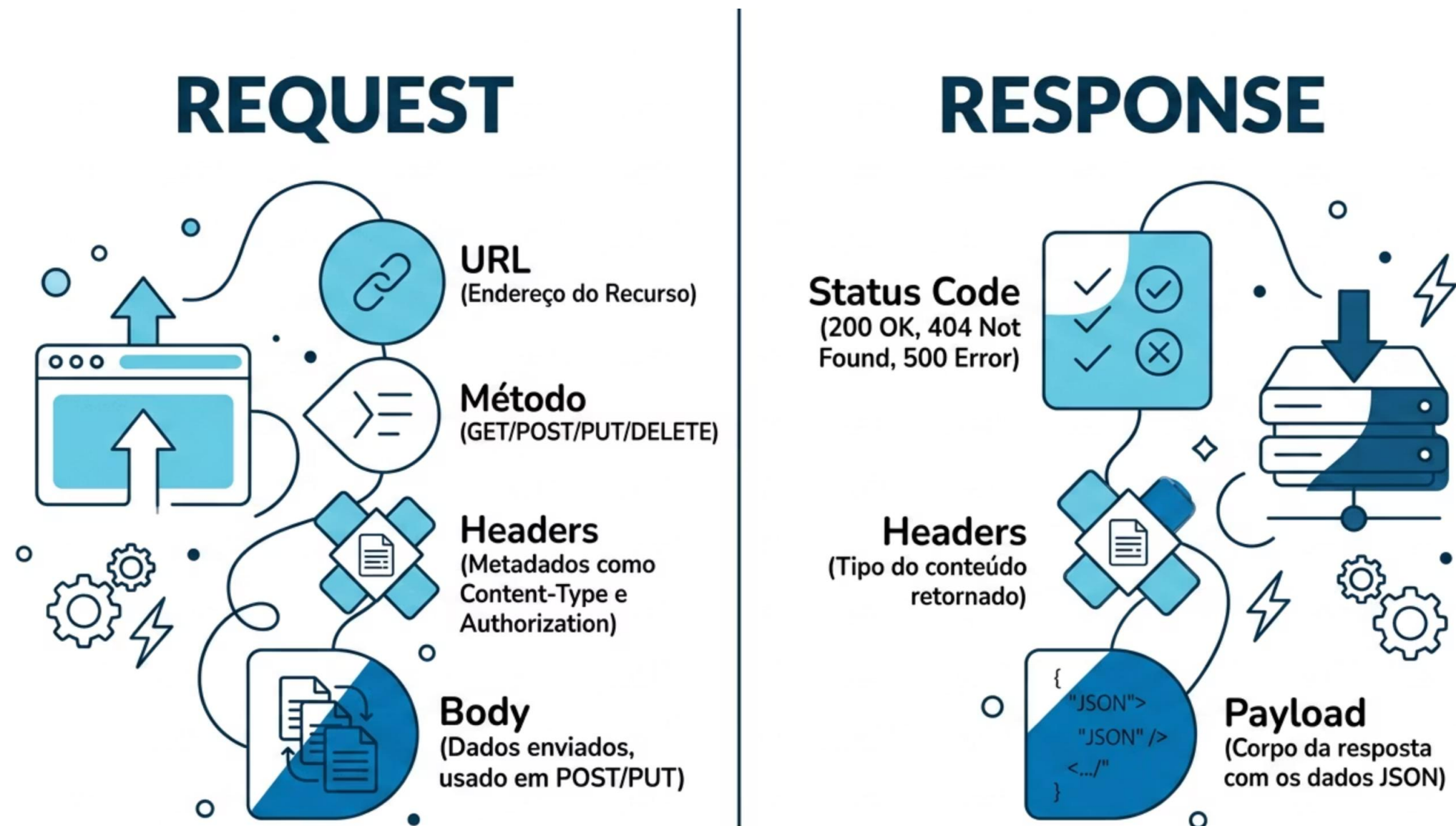
🛒 Compras Online

Dados de cartão e informações pessoais exigem canal seguro.

🔒 Login de Sistemas

Credenciais (usuário e senha) não podem ser interceptadas.

Estrutura de uma Requisição e Resposta



Toda comunicação HTTP segue este padrão: o cliente monta uma **requisição estruturada** e o servidor devolve uma **resposta padronizada**. Entender esta estrutura é fundamental para trabalhar com APIs em Angular.

API ViaCEP — Um Exemplo Real

A **API ViaCEP** é um serviço público gratuito que retorna dados de endereço a partir de um CEP. É exatamente o tipo de integração da nossa Situação de Aprendizagem.

Endpoint da Requisição

O cliente (front-end Angular) faz uma requisição **GET** para:

```
https://viacep.com.br/ws/01001000/json/
```

Nenhum dado é enviado no corpo — apenas o CEP está na URL.
O servidor interpreta, consulta sua base e responde.

Resposta JSON Retornada

```
{  
  "cep": "01001-000",  
  "logradouro": "Praça da Sé",  
  "bairro": "Sé",  
  "localidade": "São Paulo",  
  "uf": "SP"  
}
```

- ✔ **Quem pediu?** O front-end (navegador/Angular). **Quem respondeu?** O servidor da ViaCEP. **Formato recebido?** JSON — leve, legível e fácil de processar com JavaScript.

Entendendo o JSON da Resposta

O formato **JSON** (JavaScript Object Notation) é o padrão mais utilizado para troca de dados em APIs REST modernas. É estruturado em pares **chave: valor** e é nativo no ecossistema JavaScript/TypeScript.

Estrutura do Retorno ViaCEP

Chave	Valor	Significado
cep	"01001-000"	CEP formatado
logradouro	"Praça da Sé"	Nome da rua/prça
bairro	"Sé"	Nome do bairro
localidade	"São Paulo"	Cidade
uf	"SP"	Estado (UF)

Por que JSON?

- Leve e compacto para tráfego em rede
- Legível por humanos e máquinas
- Nativo em JavaScript/TypeScript
- Suportado por todas as linguagens modernas
- Padrão de mercado para APIs REST

 Em Angular, o `HttpClient` converte automaticamente o JSON da resposta em objetos TypeScript.

Vamos Pensar Juntos

 Onde ficam armazenados esses dados?

Os dados do CEP ficam em servidores externos (ViaCEP, Correios). O front-end não armazena — apenas exibe temporariamente o que recebeu.

 Toda aplicação web usa APIs?

Praticamente todas as aplicações modernas utilizam APIs. Mesmo sites simples frequentemente consomem serviços externos de mapas, pagamento, autenticação ou conteúdo.

 Como chegam ao navegador?

Via protocolo HTTP/HTTPS: o navegador faz uma requisição, o servidor responde com os dados em formato JSON, e o JavaScript os processa e exibe.

 O navegador "conversa" diretamente com o banco de dados?

Não! O banco de dados fica isolado no servidor, protegido. A API atua como intermediária, recebendo pedidos e consultando o banco de forma controlada e segura.

Explorando Requisições Reais com DevTools

Como Abrir o DevTools

1. Abra o Chrome ou Firefox
2. Pressione **F12** ou **Ctrl+Shift+I**
3. Clique na aba **Network**
4. Acesse um site (ex: ifood.com.br)
5. Observe as requisições que aparecem

O que Investigar em Cada Requisição

Método

GET, POST ou outro?

URL

Para qual endereço foi feita?

Status

200? 404? O que indica?

Payload

Quais dados foram retornados?



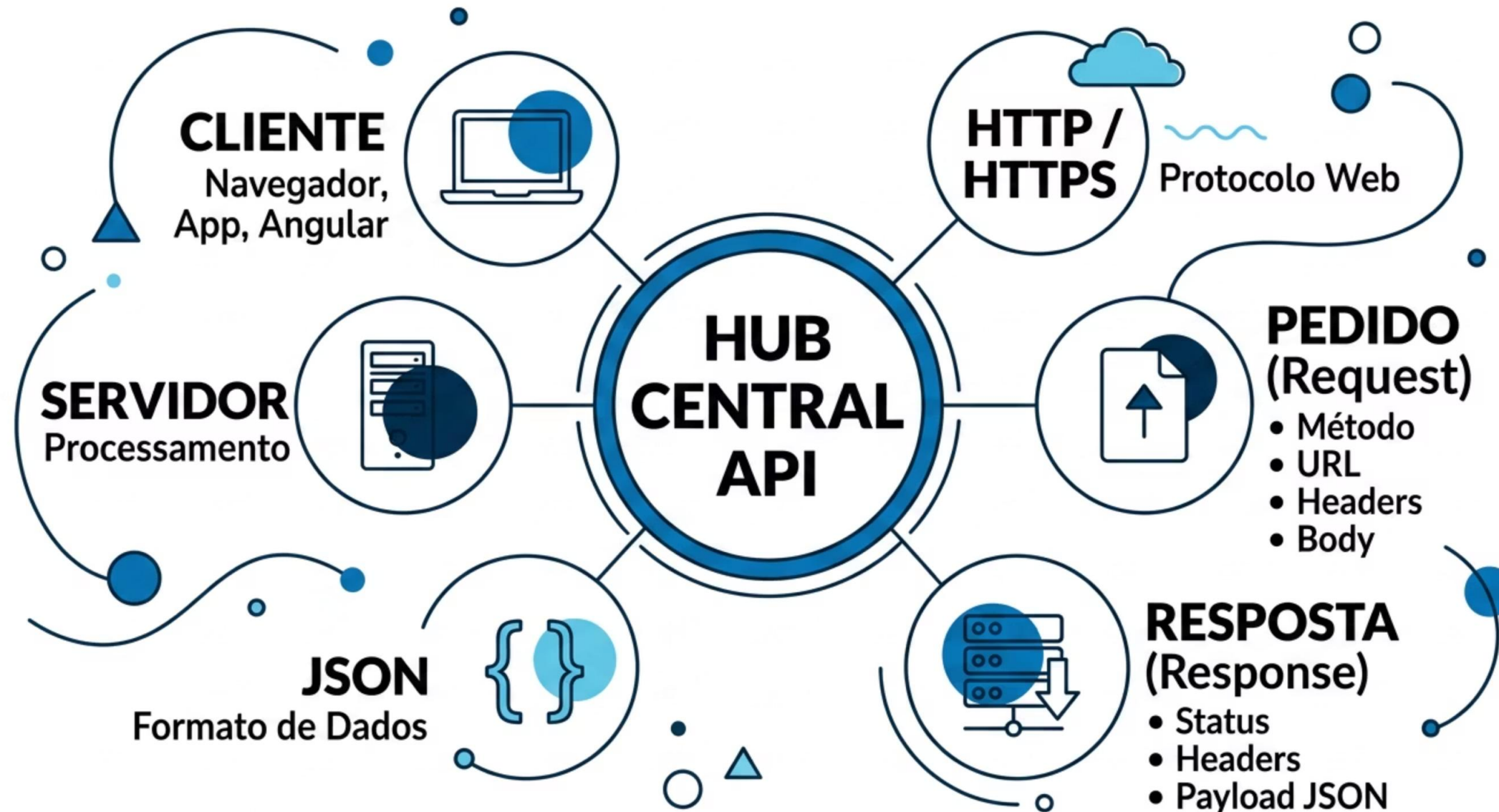
Identifique pelo menos **3 requisições diferentes** e anote: método, URL e status code de cada uma.

Fluxo Completo: CEP Automático no Sistema



Este é o ciclo completo que resolve nossa Situação de Aprendizagem. O formulário Angular **não precisa de banco de dados próprio** para endereços — ele consome um serviço externo especializado via API, obtendo dados atualizados em tempo real com segurança via HTTPS.

Resumo: Conceitos-Chave da Aula

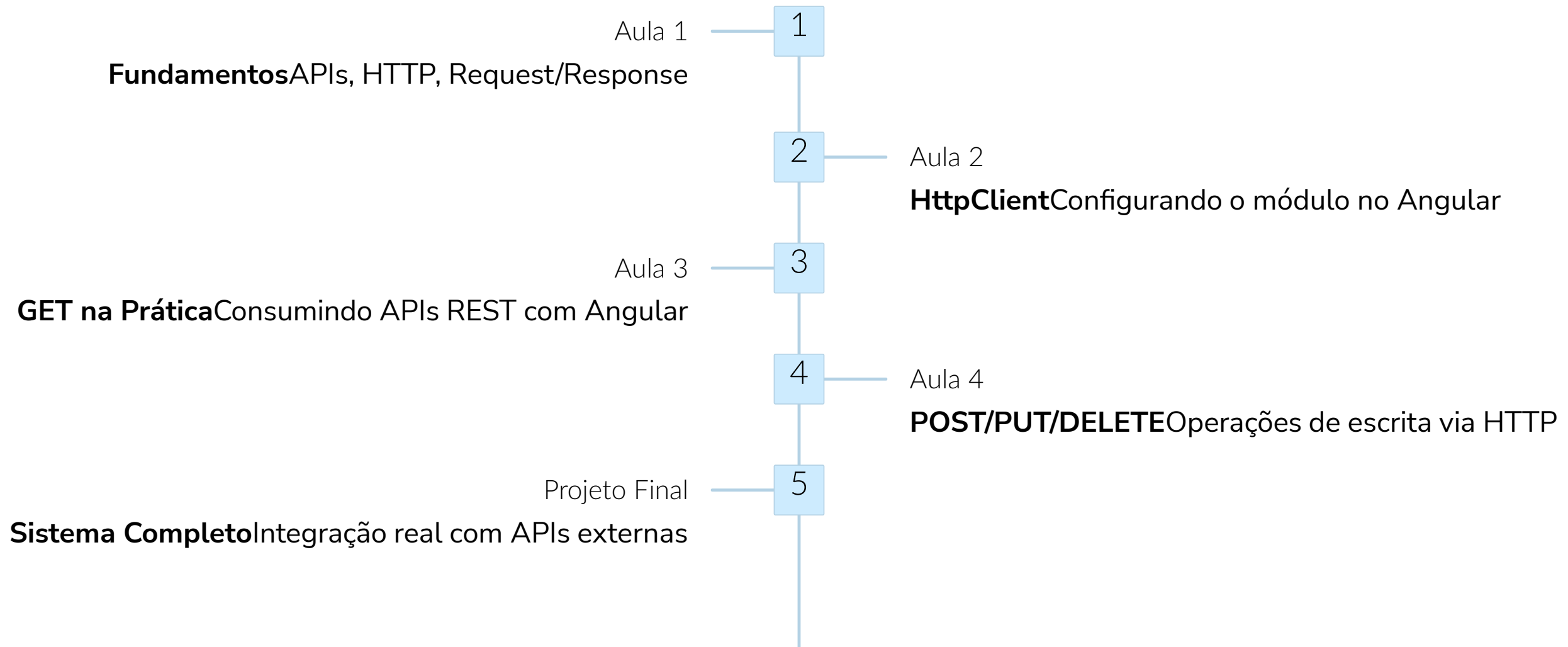


Ao Final desta Aula, Você é Capaz de...

- Explicar o papel das APIs
Descrever o que é uma API, para que serve e como ela atua como intermediária entre sistemas distintos.
- Diferenciar Cliente e Servidor
Identificar a função de cada parte na comunicação web e por que o banco de dados não é acessado diretamente.
- Compreender HTTP/HTTPS
Reconhecer o protocolo HTTP, seus métodos (GET, POST, PUT, DELETE) e a importância do HTTPS para segurança.
- Interpretar Request e Response
Identificar os componentes de uma requisição (URL, método, headers, body) e de uma resposta (status, payload JSON).
- Reconhecer APIs em aplicações reais
Observar e identificar no DevTools as requisições HTTP feitas por aplicações web do cotidiano.

Conexão com as Próximas Aulas

Os fundamentos desta aula são a base para tudo que virá na disciplina. Cada conceito será aplicado progressivamente com Angular.



✔ Na próxima aula, você irá **configurar o HttpClient no Angular** e fazer sua primeira requisição GET real para a API ViaCEP — o sistema de preenchimento automático de endereços!

Referências e Recursos Complementares

Para Aprofundar

- **MDN Web Docs** — Documentação completa de HTTP (developer.mozilla.org)
- **ViaCEP** — API pública de CEP (viacep.com.br)
- **Postman** — Ferramenta para testar APIs visualmente
- **Angular Docs** — HttpClient oficial (angular.io)
- **REST API Tutorial** — restfulapi.net

Ferramentas para Praticar

- **Chrome DevTools** — Aba Network para inspecionar requisições
- **Postman / Insomnia** — Clientes HTTP para testar endpoints
- **JSONFormatter** — Extensão para visualizar JSON no browser
- **httpbin.org** — Serviço para testar requisições HTTP

 Todas as ferramentas listadas são **gratuitas** e amplamente utilizadas no mercado de desenvolvimento.

